

PAF 2026

Examen Réparti 1: *Xenon 2: Megablast*

10 Mars 2026



Figure 1: Capture d'écran de *Xenon 2: Megablast*.

Préliminaires

Aucun barème n'est donné a priori, les copies seront évaluées sur les points suivants :

- qualité de la programmation fonctionnelle,
- maitrises des spécificités de Haskell vues en cours et en TD,
- pertinence de la modélisation : respect des principes vus au TD3, écriture systématique de propriétés (invariants, post-conditions), preuves et tests.

Le langage du partiel est le *Haskell* utilisé en cours et en TD. Les petites erreurs de syntaxe ne sont pas pénalisées.

Consigne : Les partie 1 et 2 sont guidées, les parties suivantes ne le sont pas et évaluent les compétences de modélisation travaillées en cours, en TD et en TME. Les spécifications de ces parties sont **volontairement incomplètes**.

Description du jeu

Sorti en 1989, *Xenon 2: Megablast* (par *The Bitmap Brothers*) propose un jeu de tir à défilement vertical (*vertical scrolling shooter* ou *shoot'em up*) dans lequel la joueuse dirige un vaisseau spatial qui doit éliminer ses ennemis en évitant les tirs et les collisions.

Le vaisseau de la joueuse peut se déplacer sur l'écran (vers le haut, le bas, la gauche la droite) pour éviter les collisions avec des ennemis, des obstacles ou les deux murs présents à gauche et à droite de l'écran.

Le jeu comporte un défilement (*scrolling*) : à intervalle réguliers (plusieurs fois par seconde) les obstacles, les murs et le décor au fond de l'écran se décalent vers le bas, donnant l'illusion (par parallaxe) que le vaisseau de la joueuse est constamment en mouvement vers le haut.

En plus de se déplacer, le vaisseau de la joueuse est capable de tirer des projectiles qui détruisent les ennemis quand ils entrent en collision avec eux. Les ennemis eux-mêmes peuvent tirer des projectiles sur le vaisseau de la joueuse. A chaque collision avec un ennemi ou un projectile, la barre de vie de la joueuse décroît. Quand elle est vide, le nombre d'essais de la joueuse (initialement à 2) décroît et la joueuse recommence le jeu à un point de passage récent. Le jeu s'arrête (*game over*) quand la joueuse n'a plus d'essai disponible. Le jeu comporte en outre : cinq niveaux, avec des ennemis plus difficiles à vaincre (*boss*) à la fin de chacun d'eux, des bonus lâchés par certains ennemis à leur mort qui améliorent la vaisseau de la joueuse (tir plus rapide, tir dans d'autres direction) quand celui-ci les récupère (en passant dessus avec son vaisseau), la possibilité pour la joueuse d'inverser momentanément le défilement de l'écran, la possibilité de jouer à deux simultanément, un système de score.

Dans l'examen, on se concentre sur les fonctions de bases du jeu: défilement, déplacement, vaisseau de la joueuse, ennemis, projectiles, ...

1 Objets mobiles

L'*écran* est la zone de jeu, définie par sa hauteur et sa largeur, donnés en *pixels*. les coordonnées (0,0) caractérisent le pixel en bas à gauche de l'écran, (0,1) celui juste au-dessus, (1,0) celui juste à sa droite.

Pour gérer les collisions, chaque objet du jeu possède une *hitbox*, c'est-à-dire un ensemble de pixels qui représentent la surface totale couverte par l'objet. Deux objets entrent en collision si l'intersection de leurs hitbox est non-vide.

Dans un premier temps, on considère que les *Hitbox* peuvent seulement être :

- des points définis par leurs coordonnées,
- des disques définis par les coordonnées de leur centre et leur rayon (strictement positif),
- des rectangles définis par les coordonnées de leur sommet en bas à gauche, leur largeur (strictement positive) et leur hauteur (strictement positive).
- des composés de plusieurs (au moins deux) *Hitbox* définis par la liste de leur *Hitbox* composantes.

```
data Hitbox = Point Int Int
            | Disque Int Int Int
            | Rectangle Int Int Int Int
            | Composee [Hitbox]
            deriving (Eq, Show)
```

Question 1.1 Ecrire un *invariant* pour les *Hitbox*

Question 1.2 Ecrire des *constructeurs intelligents* pour les trois premières formes de *Hitbox*.

La fonction de collision décide quand deux *Hitbox* se touchent (c'est-à-dire, quand l'intersection de leurs pixels est non-vide).

Par exemple, deux hitbox qui sont toutes les deux des points entrent en collision seulement si les points sont superposés (si ils ont les mêmes coordonnées).

```
collision :: Hitbox -> Hitbox -> Bool
collision (Point x1 y1) (Point x2 y2) = x1 == y1 && x2 == y2
collision _ _ = undefined
```

Question 1.3 Compléter le code de `collision` pour traiter les cas de collisions suivants :

1. entre deux rectangles,
2. entre un point et un cercle, en utilisant `sqrt :: Float -> Float`,
3. entre une hitbox composée et une autre hitbox.

On veut montrer "si une hitbox $h1$ est composée de deux points et si une hitbox $h2$ décrite comme un point entre en collision avec $h1$, alors $h2$ est un des deux points de $h2$ ".

Question 1.4 Ecrire une propriété décrivant cette proposition.

Question 1.5 Donner un jeu de tests *HSpec* pour cette propriété.

Question 1.6 Prouver cette propriété (expliciter les équations utilisées à chaque étape de la preuve).

2 Murs

On souhaite rajouter aux hitbox les "murs infinis" présents sur les bords de l'écran.

Un mur est défini par une séquence **infinie** de coordonnées d'ordonnées strictement croissantes.

```
data Hitbox = ...  
  | MurGauche [(Int, Int)]
```

La hitbox d'un mur gauche est l'ensemble des pixels situés à gauche d'un segment composé de deux points consécutifs de la liste.

Par exemple la séquence $(0, 0), (5, 10), (0, 20), (5, 30), (0, 40), (5, 50), \dots$ définit un mur "en dent de scie".

Le point $(1, 8)$ entre en collision avec ce mur (il est à gauche du segment reliant $(0, 0)$ et $(5, 10)$).

Le point $(5, 19)$ n'entre pas en collision avec ce mur (il est à droite du segment reliant $(5, 10)$ et $(0, 20)$).

Question 2.1 Ecrire une définition de ce mur gauche en dent de scie.

Question 2.2 Ecrire une définition d'un mur gauche vertical situé à 10 pixels du bord de l'écran.

Question 2.3 Ajouter à la fonction `collision` un cas traitant la collision entre un point et un mur gauche.

3 Obstacles, Projectiles, Tour

Les objets présents à l'écran comportent les vaisseaux des joueuses, les ennemis, les obstacles, les projectiles. Les obstacles sont uniquement définis par une hitbox. Les projectiles comportent une hitbox, et une direction, qui indique dans quel sens ils se déplacent. A la fin de chaque tour de jeu, on gère le déplacement des objets. Les obstacles peuvent être déplacés vers le bas (c'est le principe du "défilement") et les projectiles avancent dans leur direction.

Pour pouvoir gérer des vitesses différentes (entre les projectiles, le défilement des obstacles et le déplacement du vaisseau et des ennemis), on fait en sorte qu'un tour de jeu soit "plus rapide" que le défilement. C'est-à-dire que les obstacles ne descendent pas de 1 pixel à chaque tour, mais de 1 pixel tous les x tours, avec x un paramètre du moteur de jeu.

Les projectiles peuvent avoir des vitesses différentes. La vitesse d'un projectile est définie par le nombre de tour qu'il attend avant d'avancer dans sa direction (intuitivement plus cette valeur est élevée, plus le projectile sera lent).

Question 3 Définir un type pour les objets présent à l'écran comportant initialement des projectiles et des obstacles.

Définir un invariant pour ce type, des constructeurs intelligents, des opérations de déplacement avec pré-conditions et post-conditions et des opérations de "fin de tour" (qui décrivent comment l'objet évolue à la fin d'un tour) avec pré-conditions et post-conditions.

Donner des exemples, des jeux de tests, des preuves, ...

4 Joueuses

Les vaisseaux des joueuses ont comme caractéristiques (entre autres) une hitbox, un nombre de point de vie, un nombre d'essai restant.

Les vaisseaux réagissent à des commandes : déplacement dans les quatres directions, tir. La vitesse de ces vaisseaux peut évoluer au cour du jeu.

Quand un vaisseau d'une joueuse reçoit un ordre de tir, il crée un projectile qui se déplace vers le haut de l'écran.

Le vaisseau perd des points de vie s'il entre en collision avec un projectile, et il est "repoussé" par les murs et les obstacles.

Question 4 Modifier le type de la question précédente pour prendre en compte les vaisseaux des joueuses. Ecrire définitions, invariants, constructeurs intelligents, opérations avec préconditions et postconditions.

5 Ennemis

Les ennemis se déplacent selon une "intelligence artificielle" donnée sous forme d'oracle qui indique la prochaine action de l'ennemi (déplacement, tir, ...) scriptée (par exemple, l'ennemi suit une trajectoire prédéfinie) ou, éventuellement, pseudo-aléatoire (dépendant d'un entier, qui sera donné au cours du jeu par un générateur de pseudo aléatoire).

Les ennemis ont des points de vie, comme le vaisseau de la joueuse, qu'ils perdent en entrant en collision avec les tirs du vaisseau de la joueuse (on pourra introduire un système séparant les tirs des joueuses de ceux des ennemis).

Question 4 Modifier le type de la question 3. pour prendre en compte les ennemis. Ecrire définitions, invariants, constructeurs intelligents, opérations avec préconditions et postconditions.

6 Moteur

A tout moment, le jeu maintient une structure de données listant tous les objets présents à l'écran. A la fin de chaque tour global, il fait avancer chaque objet d'un tour (impliquant potentiellement son déplacement, ou sa destruction).

Le moteur du jeu dispose en outre d'un *script* qui indique les prochains évènements (apparition ou disparition d'ennemis, d'obstacles, de projectiles, ...)

Question 5 Ecrire un type pour le moteur de jeu prenant en compte ces fonctionnalités. Ecrire définitions, invariants, constructeurs intelligents, opérations avec préconditions et postconditions.